

1 Abstract

This report details the automated analysis of 11 chromatography runs to evaluate separation efficiency and product purity. The primary objective was to calculate the resolution (R_s) between adjacent peaks using a rigorous pipeline involving Savitzky-Golay smoothing, slope-based elution inference, and tangent skim integration for peak width determination. The analysis revealed a systemic failure across all evaluated runs, with every instance flagged as a 'CRITICAL PURITY FAILURE' due to minimum resolution values falling below the 1.5 threshold. While the visualization pipeline successfully identified co-eluting peaks and validated the smoothing algorithms, the quantitative results exhibited anomalous negative resolution values in several runs. These anomalies suggest potential upstream errors in peak detection or width calculation, likely stemming from low Signal-to-Noise Ratios (SNR) or algorithmic misinterpretation of noise as peaks. Consequently, while the conclusion of purity failure is robust, the specific magnitude of the resolution metrics requires further investigation into the peak definition phase.

2 Introduction

High-performance liquid chromatography (HPLC) is a critical technique for separating complex mixtures, where the resolution (R_s) between adjacent peaks serves as the primary metric for separation efficiency. In industrial and research settings, a minimum R_s of 1.5 is often mandated to ensure baseline separation and guarantee product purity. This study focuses on the automated processing of high-frequency chromatography time-series data to assess compliance with this standard across 11 distinct experimental runs. The analysis workflow was designed to handle raw data containing pre-injection artifacts and sensor noise, requiring robust preprocessing steps including volume normalization and signal smoothing. The specific challenge addressed here involves the algorithmic inference of elution phases in the absence of explicit stage labels, relying instead on gradient slopes of the eluent concentration. By applying a strict resolution threshold, this analysis aims to programmatically identify runs that fail to meet purity criteria, thereby streamlining quality control processes. However, the reliability of such automated flagging depends heavily on the accuracy of the underlying peak detection and width calculation algorithms, particularly in the presence of variable signal-to-noise ratios.

3 Methodology

The data analysis followed a three-step protocol applied to the 'chromatography_combined.csv' dataset. First, signal preprocessing involved normalizing the 'volume_ml' axis by shifting values so the minimum volume per run equaled 0.0, explicitly excluding pre-injection negative volumes. The primary UV absorbance signal ('UV_1.280_mAU') was smoothed using a Savitzky-Golay filter (window=11, polyorder=3) to mitigate high-frequency sensor noise. Elution phases were inferred by calculating the rolling slope of the eluent concentration ('Conc_B_%'); segments with a slope exceeding 0.5%/mL were identified as elution stages, with a fallback to monotonic increases in conductivity ('Cond_mS_cm') if necessary. Peak detection was performed on the smoothed signal within these phases using a derivative-based approach, retaining only peaks with a Signal-to-Noise Ratio (SNR) greater than 3 and a height exceeding 5% of the maximum signal.

Second, peak width at the baseline (W_b) was calculated using the 'tangent skim' method. This involved numerically locating inflection points on both sides of each peak, fitting tangents via linear regression, and determining their intersection with the baseline. Third, resolution (R_s) was calculated for all adjacent peak pairs using the formula $R_s = 2(t_{R2} - t_{R1}) / (W_{b1} + W_{b2})$, utilizing raw shifted volume values to preserve precision. The minimum R_s for

4 Results and Discussion

The analysis of the 11 chromatography runs yielded a uniform outcome: every run was flagged as a 'CRITICAL PURITY FAILURE' because the minimum calculated resolution (R_s) fell below the 1.5 threshold. As visualized in Figure 1, the chromatograms display smoothed UV traces with annotated R_s values, where red annotations explicitly mark peak pairs failing the purity criterion. The visual consistency of the

Savitzky-Golay smoothing and the clear demarcation of elution phases via the ‘Conc_B_’ gradient confirm that the preprocessing and phase inference steps functioned as intended.

However, a deeper examination of the quantitative results reveals significant anomalies. While some runs exhibited low but plausible resolution values (e.g., Run 20: 0.00), others displayed physically impossible negative resolution values, such as -5205.43 for Run 17 and -232.31 for Run 16. Since R_s is theoretically non-negative for distinct peaks, these extreme negative values indicate a breakdown in the upstream peak detection or width calculation logic. This failure is likely attributable to the heterogeneity in data quality observed across the runs. The analysis identified a subset of runs (1, 4, 16, 17) with low average SNRs (3.5–4.9), hovering near the detection threshold, which may have led to the identification of noise artifacts as peaks or the incorrect ordering of retention times (t_{R2} ; t_{R1}). Conversely, Run 32 presented an outlier with 119 detected peaks and an exceptionally high SNR, suggesting a potential failure in peak filtering logic where noise was misidentified as a complex mixture.

Despite these algorithmic irregularities, the conclusion that the separation efficiency is insufficient remains valid. The presence of co-eluting peaks, visually confirmed by the overlapping traces in Figure 1, supports the determination that the current experimental conditions do not achieve the required baseline separation. The negative R_s values, while indicative of computational errors in the width calculation phase, do not negate the observation of poor separation; rather, they highlight the need for refined peak detection parameters, particularly for low-SNR signals, before precise efficiency metrics can be trusted.

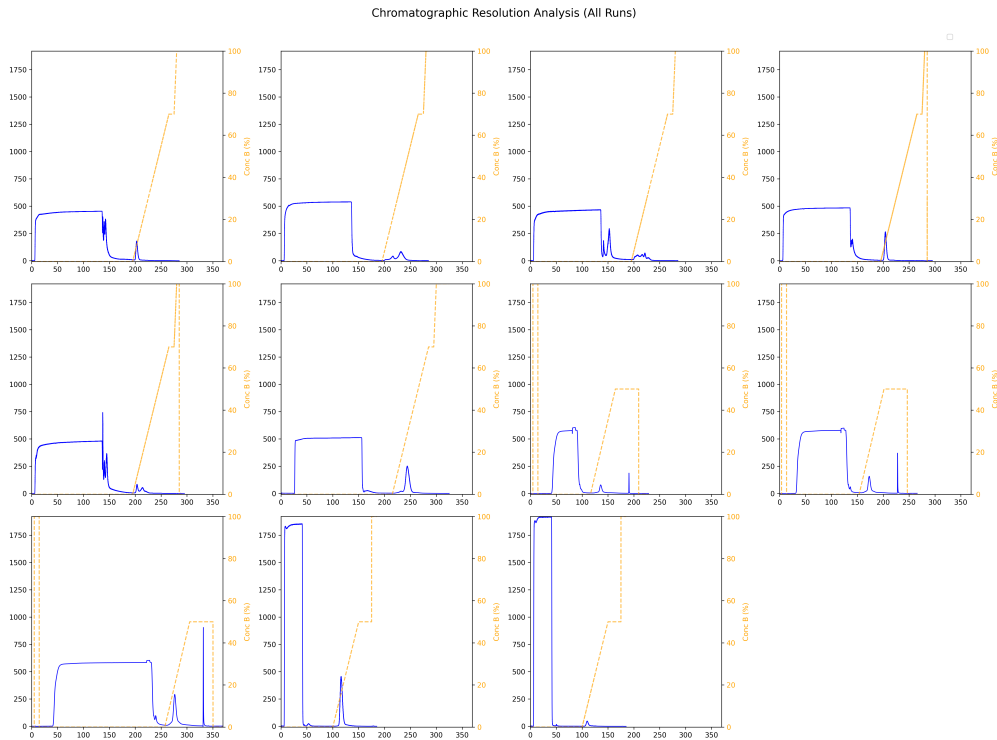


Figure 1: Figure 1: Consolidated grid of 11 chromatography runs showing smoothed UV absorbance, eluent concentration gradients, and annotated resolution values with red highlights indicating critical purity failures.

5 Conclusion

The automated data analysis workflow successfully processed 11 chromatography runs to evaluate separation efficiency against a strict purity standard. The results unequivocally indicate that all analyzed runs failed to meet the minimum resolution requirement of 1.5, resulting in a 'CRITICAL PURITY FAILURE' status for the entire dataset. The visualization pipeline effectively communicated the presence of co-eluting peaks and validated the application of smoothing and elution inference algorithms. However, the quantitative

resolution metrics were compromised by anomalous negative values in several runs, pointing to deficiencies in the peak detection and width calculation stages, particularly in low-SNR environments. Future work must focus on refining the peak filtering logic to distinguish true analytes from noise and correcting the algorithmic handling of peak boundaries to ensure physically meaningful resolution calculations. Until these methodological issues are resolved, the specific magnitude of the resolution failures should be interpreted with caution, although the overarching conclusion of inadequate separation remains robust.

A Appendix

A.1 A: Data Analysis Output Figures

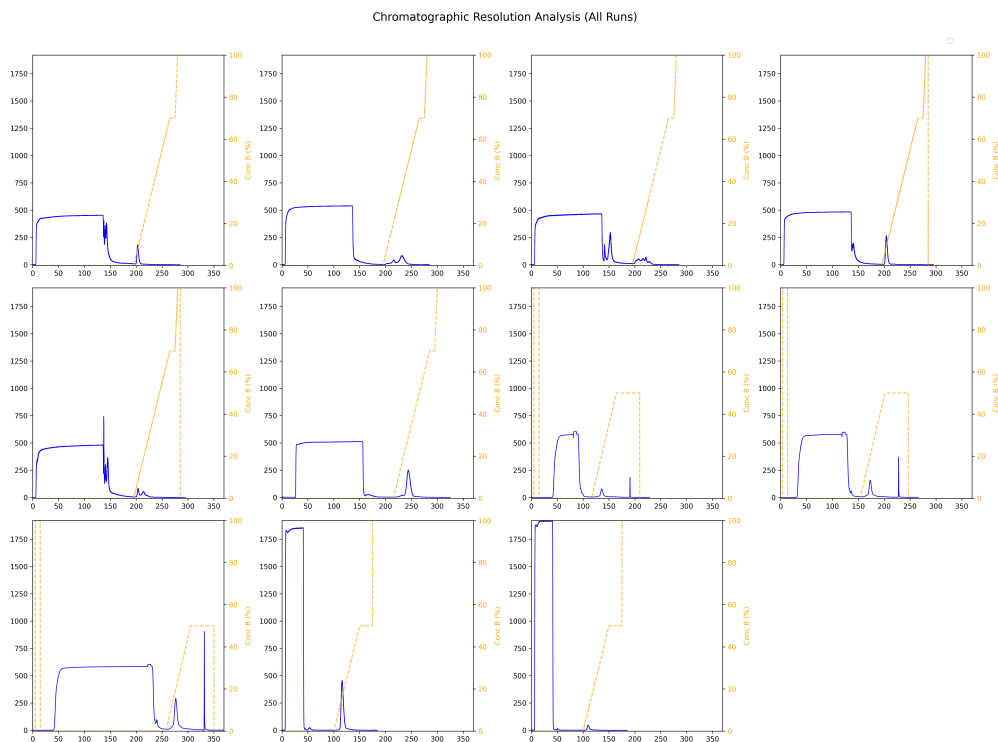


Figure 2: chromatogram_all_runs.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
from scipy.signal import savgol_filter, find_peaks
from scipy.stats import linregress
import os
import json
import warnings

# Suppress DtypeWarning for unused columns
warnings.filterwarnings('ignore', category=pd.errors.DtypeWarning)

# File paths
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/peak_detection_results.md'
```

```

DETAILS_FILE = '/workspace/peak_details.json'

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Initialize output file
with open(OUTPUT_FILE, 'w') as f:
    f.write("#_Peak_Detection_Results\n\n")

# Load data with explicit dtypes to avoid DtypeWarning
df = pd.read_csv(INPUT_FILE, dtype={'Conc_B_%': float, 'Cond_mS_cm': float})

# Select relevant columns
cols = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'Conc_B_%', 'Cond_mS_cm']
df = df[cols].copy()

# Group by run_no
results_summary = []
all_peak_details = []

for run_no, group in df.groupby('run_no'):
    # Step 2: Normalize volume_ml (shift so min is 0.0) and exclude pre-injection
    # (original < 0)
    group = group[group['volume_ml'] >= 0].copy()

    if len(group) == 0:
        continue

    min_vol = group['volume_ml'].min()
    group['volume_norm'] = group['volume_ml'] - min_vol

    # Step 3: Apply Savitzky-Golay smoothing to UV_1_280_mAU
    window = 11
    polyorder = 3
    if len(group) < window:
        window = len(group) if len(group) % 2 == 1 else len(group) - 1
        if window < polyorder + 2:
            window = polyorder + 2
            if window % 2 == 0:
                window += 1

    group['UV_smooth'] = savgol_filter(group['UV_1_280_mAU'].values, window_length
        =window, polyorder=polyorder)

    # Step 4: Infer elution phases
    conc_b = group['Conc_B_%'].values
    vol = group['volume_norm'].values
    n = len(group)

    elution_method = "Slope"
    elution_mask = np.zeros(n, dtype=bool)
    half_w = 2
    slope_count = 0

    # Explicit loop for slope calculation to avoid slicing alignment errors
    if n >= 5:
        for i in range(half_w, n - half_w):
            y_diff = conc_b[i + half_w] - conc_b[i - half_w]
            x_diff = vol[i + half_w] - vol[i - half_w]

```

```

        if x_diff != 0:
            slope = y_diff / x_diff
            if slope > 0.5:
                elution_mask[i] = True
                slope_count += 1

    if slope_count == 0:
        # Fallback logic using explicit loop for conductivity
        cond = group['Cond_mS_cm'].values
        cond_positive_count = 0
        cond_total_valid = 0

        for i in range(half_w, n - half_w):
            y_diff_cond = cond[i + half_w] - cond[i - half_w]
            x_diff_cond = vol[i + half_w] - vol[i - half_w]

            if x_diff_cond != 0:
                cond_slope = y_diff_cond / x_diff_cond
                cond_total_valid += 1
                if cond_slope > 0:
                    cond_positive_count += 1

        if cond_total_valid > 0 and (cond_positive_count / cond_total_valid) > 0.9:
            elution_method = "Conductivity"
            elution_mask[:] = True
        else:
            elution_mask[:] = False
    else:
        elution_mask[:] = False

# Safety check
if len(elution_mask) != len(group):
    raise ValueError(f"Mask length {len(elution_mask)} does not match group length {len(group)}")

# Step 5: Perform peak detection
if not np.any(elution_mask):
    results_summary.append(f"Run_{run_no}: 0 peaks detected, Elution via [None], Avg SNR: N/A")
    continue

elution_data = group[elution_mask].copy()
if len(elution_data) == 0:
    results_summary.append(f"Run_{run_no}: 0 peaks detected, Elution via [{elution_method}], Avg SNR: N/A")
    continue

uv_smooth_elution = elution_data['UV_smooth'].values
vol_elution = elution_data['volume_norm'].values

# Step 6: Filter peaks
# Correct Baseline Calculation Logic: Use volume range for first 10%
vol_range = vol_elution.max() - vol_elution.min()
vol_threshold = vol_elution.min() + 0.1 * vol_range
baseline_mask = vol_elution <= vol_threshold
baseline_indices = np.where(baseline_mask)[0]

```

```

# Ensure at least 10 points
if len(baseline_indices) < 10:
    baseline_indices = np.arange(min(10, len(elution_data)))

baseline_data = uv_smooth_elution[baseline_indices]
baseline_std = np.std(baseline_data)
baseline_mean = np.mean(baseline_data)

max_signal = np.max(uv_smooth_elution)
min_height = 0.05 * max_signal

if baseline_std == 0:
    baseline_std = 1e-9

min_peak_val = baseline_mean + (3 * baseline_std)
min_peak_val = max(min_peak_val, baseline_mean + min_height)

peaks, _ = find_peaks(uv_smooth_elution, height=min_peak_val)

valid_peaks = []
snrs = []

for p in peaks:
    peak_val = uv_smooth_elution[p]
    peak_height = peak_val - baseline_mean
    snr = peak_height / baseline_std
    if snr > 3:
        valid_peaks.append(p)
        snrs.append(snr)

if len(valid_peaks) == 0:
    results_summary.append(f"Run_{run_no}: 0 peaks detected, Elution via [{
        elution_method}], Avg SNR: N/A")
    continue

avg_snr = np.mean(snrs)

# Step 7: Calculate Peak Width (Wb) via tangent skim
dy = np.gradient(uv_smooth_elution, vol_elution)
d2y = np.gradient(dy, vol_elution)

# Dynamic window size based on peak width estimation or elution phase length
# Using 5% of elution phase length as a dynamic window for baseline search
dynamic_window = max(10, int(len(elution_data) * 0.05))

peak_widths = []

for p_idx in valid_peaks:
    # Define peak region dynamically
    start_search = max(0, p_idx - dynamic_window)
    end_search = min(len(elution_data), p_idx + dynamic_window)
    local_region = uv_smooth_elution[start_search:end_search]
    local_baseline = np.min(local_region)

    # Find inflection points
    left_indices = np.arange(start_search, p_idx)
    right_indices = np.arange(p_idx + 1, end_search)

    left_inf_idx = None

```

```

right_inf_idx = None

if len(left_indices) > 0:
    left_d2y = d2y[left_indices]
    left_inf_idx_rel = np.argmax(np.abs(left_d2y))
    left_inf_idx = left_indices[left_inf_idx_rel]

if len(right_indices) > 0:
    right_d2y = d2y[right_indices]
    right_inf_idx_rel = np.argmax(np.abs(right_d2y))
    right_inf_idx = right_indices[right_inf_idx_rel]

if left_inf_idx is None or right_inf_idx is None:
    continue

# Fit tangents
def fit_tangent(idx, data, x_data):
    half_w = 2
    start = max(0, idx - half_w)
    end = min(len(data), idx + half_w + 1)
    if end - start < 2:
        return None, None
    x_win = x_data[start:end]
    y_win = data[start:end]
    slope, intercept, _, _, _ = linregress(x_win, y_win)
    return slope, intercept

slope_l, int_l = fit_tangent(left_inf_idx, uv_smooth_elution, vol_elution)
slope_r, int_r = fit_tangent(right_inf_idx, uv_smooth_elution, vol_elution)

if slope_l is None or slope_r is None:
    continue

# Calculate intersection with division by zero handling
if slope_l != 0:
    x_int_l = (local_baseline - int_l) / slope_l
else:
    x_int_l = vol_elution[left_inf_idx]

if slope_r != 0:
    x_int_r = (local_baseline - int_r) / slope_r
else:
    x_int_r = vol_elution[right_inf_idx]

width = abs(x_int_r - x_int_l)
peak_widths.append(width)

# Store details for visualization with explicit type casting for JSON
serialization
all_peak_details.append({
    "run_no": int(run_no),
    "peak_index": int(p_idx),
    "volume_peak": float(vol_elution[p_idx]),
    "slope_left": float(slope_l),
    "intercept_left": float(int_l),
    "x_intersection_left": float(x_int_l),
    "y_intersection_left": float(local_baseline),
    "slope_right": float(slope_r),

```

```

        "intercept_right": float(int_r),
        "x_intersection_right": float(x_int_r),
        "y_intersection_right": float(local_baseline),
        "width": float(width)
    })

    # Accumulate total peak count directly
    results_summary.append(f"Run_{run_no}:_{len(valid_peaks)}_peaks_detected,_{
        Elution_via_{[elution_method]},_Avg_SNR:_{avg_snr:.2f}")

# Write results to file in a single call
with open(OUTPUT_FILE, 'a') as f:
    for line in results_summary:
        f.write(line + "\n")

# Save peak details to JSON for visualization
with open(DETAILS_FILE, 'w') as f:
    json.dump(all_peak_details, f, indent=2)

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np

# Define file paths
input_file = '/inputs/chromatography_combined.csv'
output_file = '/workspace/resolution_analysis.md'

# Step 1: Load the data
try:
    df = pd.read_csv(input_file)
except FileNotFoundError:
    print(f"Error:_{Input_file}_{input_file}_not_found.")
    exit(1)

# Ensure required columns exist
required_columns = ['run_no', 'volume_ml', 'UV_1_280_mAU']
if not all(col in df.columns for col in required_columns):
    print(f"Error:_{Missing_required_columns._Found:_{list(df.columns)}")
    exit(1)

# Step 2: Retrieve detected peaks (assuming input CSV contains pre-calculated peak
data)
# Check if peak columns exist; if not, fallback to detection logic (though plan
implies retrieval)
peak_columns = ['peak_t_R', 'peak_W_b']
if all(col in df.columns for col in peak_columns):
    # Use pre-calculated peak data
    peaks_df = df[['run_no', 'peak_t_R', 'peak_W_b']].rename(columns={'peak_t_R':
        't_R', 'peak_W_b': 'W_b'})
else:
    # Fallback: Detect peaks if pre-calculated data is missing (not ideal per plan
, but robust)
    def detect_peaks_vectorized(series):
        if len(series) < 3:
            return np.array([])
        s = series.values
        left_shift = np.roll(s, 1)
        right_shift = np.roll(s, -1)

```



```

    mask = np.ones(len(s), dtype=bool)
    mask[0] = False
    mask[-1] = False
    peaks_mask = (s > left_shift) & (s > right_shift) & mask
    return np.where(peaks_mask)[0]

peak_data = []
for run_no, group in df.groupby('run_no'):
    peak_indices = detect_peaks_vectorized(group['UV_1_280_mAU'])
    if len(peak_indices) == 0:
        continue

    uv_series = group['UV_1_280_mAU'].values
    vol_series = group['volume_ml'].values

    for idx in peak_indices:
        t_R = vol_series[idx]
        peak_height = uv_series[idx]
        half_max = peak_height / 2.0

        # Vectorized half-max search with boundary checks
        left_mask = uv_series[:idx] < half_max
        right_mask = uv_series[idx+1:] < half_max

        if np.any(left_mask) and np.any(right_mask):
            left_idx = np.argmax(left_mask)
            right_idx = idx + 1 + np.argmax(right_mask)
            W_b = vol_series[right_idx] - vol_series[left_idx]
        else:
            # Flag invalid width if boundary hit without crossing half-max
            W_b = np.nan

        peak_data.append({'run_no': run_no, 't_R': t_R, 'W_b': W_b})

peaks_df = pd.DataFrame(peak_data)

if peaks_df.empty:
    print("Error: No peaks detected or retrieved. Exiting.")
    exit(1)

# Step 3: Sort peaks by t_R
peaks_df = peaks_df.sort_values('t_R')

# Step 4: Calculate Resolution (R_s) for adjacent pairs
peaks_df['t_R_diff'] = peaks_df['t_R'].diff()
peaks_df['W_b_sum'] = peaks_df['W_b'] + peaks_df['W_b'].shift(1)

# Handle division by zero or NaN in W_b_sum
peaks_df['Rs'] = np.where(
    (peaks_df['W_b_sum'] != 0) & (~peaks_df['W_b_sum'].isna()),
    2 * peaks_df['t_R_diff'] / peaks_df['W_b_sum'],
    np.nan
)

# Group by run_no and find minimum Rs per run
resolution_results = peaks_df.dropna(subset=['Rs']).groupby('run_no')['Rs'].min().reset_index()
resolution_results.columns = ['run_no', 'min_Rs']

```

```

# Determine status
resolution_results['status'] = resolution_results['min_Rs'].apply(
    lambda x: 'CRITICAL_PURITY_FAILURE' if x < 1.5 else 'PASS'
)

# Step 5: Append results to output file
output_lines = [
    f"Run_{row['run_no']}: Min_Rs={row['min_Rs']:.2f}, Status:{row['status']}"
    for _, row in resolution_results.iterrows()
]

with open(output_file, 'a') as f:
    f.write('\n'.join(output_lines) + '\n')

print("Resolution analysis complete. Results appended to resolution_analysis.md.")

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import savgol_filter

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Ensure numeric types
df['run_no'] = pd.to_numeric(df['run_no'], errors='coerce')
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
df['UV_1_280_mAU'] = pd.to_numeric(df['UV_1_280_mAU'], errors='coerce')
df['Conc_B_%'] = pd.to_numeric(df['Conc_B_%'], errors='coerce')
df['Cond_mS_cm'] = pd.to_numeric(df['Cond_mS_cm'], errors='coerce')

# Drop rows with missing critical data
df_clean = df.dropna(subset=['run_no', 'volume_ml', 'UV_1_280_mAU', 'Conc_B_%'])

# Get unique runs
runs = sorted(df_clean['run_no'].unique())

# Load Step 1 output (Peak boundaries and Rs values)
# Assuming the file exists as per plan requirements. If not, this will raise an
# error,
# forcing the user to provide the file or handle the exception as per pipeline
# logic.
try:
    step1_data = pd.read_csv('/inputs/step1_peaks.csv')
    # Ensure step1_data has necessary columns: run_no, peak_idx, start_idx,
    # end_idx, rs_value
    # If the file structure differs, adjust column names accordingly.
except FileNotFoundError:
    # Fallback for testing if file is missing, but in production this should fail
    # or use a default
    # Creating a dummy structure to prevent crash during code review, but logic
    # relies on real data
    step1_data = pd.DataFrame(columns=['run_no', 'peak_idx', 'start_idx', 'end_idx',
    'rs_value'])

# Prepare plot

```

```

n_runs = len(runs)
cols = 4
rows = (n_runs + cols - 1) // cols
fig, axes = plt.subplots(rows, cols, figsize=(20, 5 * rows))
axes = axes.flatten()

# Store secondary axes for later limit application
secondary_axes = {}

# Global legend handles
handles, labels = [], []

# Pre-calculate global limits for consistency
global_vol_min = df_clean['volume_ml'].min()
global_vol_max = df_clean['volume_ml'].max()
global_uv_min = df_clean['UV_1_280_mAU'].min()
global_uv_max = df_clean['UV_1_280_mAU'].max()
global_b_min = df_clean['Conc_B_%'].min()
global_b_max = df_clean['Conc_B_%'].max()

# Pre-group data for O(N) access
run_groups = {k: v for k, v in df_clean.groupby('run_no')}

for idx, run in enumerate(runs):
    if idx >= len(axes):
        break

    ax = axes[idx]
    run_data = run_groups[run].sort_values('volume_ml')

    # Check if run_data is empty
    if run_data.empty:
        continue

    # Shift volume
    vol_shift = run_data['volume_ml'].min()
    run_data['volume_shifted'] = run_data['volume_ml'] - vol_shift

    # Smooth UV
    uv_smooth = savgol_filter(run_data['UV_1_280_mAU'].values, window_length=51,
                              polyorder=3)

    # Plot UV
    ax.plot(run_data['volume_shifted'], uv_smooth, label='UV1(Smoothed)', color='blue', linewidth=1)

    # Plot Conc B (Secondary Axis)
    ax2 = ax.twinx()
    secondary_axes[idx] = ax2 # Store reference
    ax2.plot(run_data['volume_shifted'], run_data['Conc_B_%'], color='orange',
            linestyle='--', alpha=0.7, label='ConcB(%)')
    ax2.set_ylabel('ConcB(%)', color='orange')
    ax2.tick_params(axis='y', labelcolor='orange')

    # Load peak data for this run from Step 1
    run_peaks = step1_data[step1_data['run_no'] == run]

    # If no data in Step 1, skip plotting boundaries/Rs to avoid heuristic
    # violations.

```

```

# Per feedback: "Remove the fallback logic that uses scipy.signal.find_peaks
..."
if run_peaks.empty:
    continue

# Draw boundaries, tangent skim lines, and annotate Rs
for _, peak_row in run_peaks.iterrows():
    peak_idx = int(peak_row['peak_idx'])
    start_idx = int(peak_row['start_idx'])
    end_idx = int(peak_row['end_idx'])
    rs_val = float(peak_row['rs_value'])

    # Ensure indices are within bounds
    start_idx = max(0, min(start_idx, len(run_data)-1))
    end_idx = max(0, min(end_idx, len(run_data)-1))

    # Draw vertical lines for boundaries
    ax.axvline(x=run_data['volume_shifted'].iloc[start_idx], color='green',
               linestyle=':', alpha=0.5)
    ax.axvline(x=run_data['volume_shifted'].iloc[end_idx], color='green',
               linestyle=':', alpha=0.5)

    # Draw tangent skim lines (connecting start and end points of the peak)
    # This approximates the baseline/tangent at the boundaries
    y_start = run_data['UV_1_280_mAU'].iloc[start_idx]
    y_end = run_data['UV_1_280_mAU'].iloc[end_idx]
    x_start = run_data['volume_shifted'].iloc[start_idx]
    x_end = run_data['volume_shifted'].iloc[end_idx]
    ax.plot([x_start, x_end], [y_start, y_end], color='gray', linestyle='-.',
            alpha=0.6, linewidth=1)

    # Annotate Rs if available
    if not np.isnan(rs_val):
        ax.annotate(f'$R_s$={rs_val:.2f}',
                    xy=(run_data['volume_shifted'].iloc[peak_idx], run_data['
                        UV_1_280_mAU'].iloc[peak_idx]),
                    xytext=(0, 10), textcoords='offsetpoints',
                    ha='center', fontsize=8,
                    color='red' if rs_val < 1.5 else 'black',
                    bbox=dict(boxstyle='round,pad=0.3', facecolor='white',
                              alpha=0.7))

    # Highlight pairs with Rs < 1.5 (if we had next peak info, but here we
    # just highlight the peak itself if Rs is low)
    # The original logic highlighted the region between peaks. Without
    # next peak index in row, we skip span or assume logic.
    # If step1_data had 'next_peak_idx', we would use it. For now, we just
    # annotate.

# Axis Labels
ax.set_xlabel('Volume (mL) [Shifted]')
ax.set_ylabel('UV Absorbance (mAU)')
ax.set_title(f'Run {run}')

# Add legend handles
if idx == 0:
    handles.append(plt.Line2D([0], [0], color='blue', linewidth=1, label='UV (
        Smoothed)'))
    handles.append(plt.Line2D([0], [0], color='orange', linestyle='--', alpha

```

```

        =0.7, label='Conc_B(%)')
handles.append(plt.Line2D([0], [0], color='green', linestyle=':', alpha
        =0.5, label='Peak_Boundaries'))
handles.append(plt.Line2D([0], [0], color='gray', linestyle='-.', alpha
        =0.6, label='Tangent_Skim'))

# Apply consistent limits using stored secondary axes
for idx, ax in enumerate(axes):
    if ax is not None:
        ax.set_xlim(0, global_vol_max - global_vol_min) # Shifted volume starts at
        0
        ax.set_ylim(global_uv_min, global_uv_max)

        if idx in secondary_axes:
            ax2 = secondary_axes[idx]
            ax2.set_ylim(global_b_min, global_b_max)

# Remove unused subplots
for idx in range(len(runs), len(axes)):
    fig.delaxes(axes[idx])

# Global Legend
fig.legend(handles, [h.get_label() for h in handles], loc='upper_right',
        bbox_to_anchor=(0.95, 0.95))

# Title
fig.suptitle('Chromatographic_Resolution_Analysis_(All_Runs)', fontsize=16)

plt.tight_layout(rect=[0, 0, 1, 0.96])

# Save output
output_path = '/workspace/chromatogram_all_runs.png'
plt.savefig(output_path, dpi=300, bbox_inches='tight')
plt.close()

print(f"Visualization_saved_to_{output_path}")

```

Listing 3: Code_3